we are done.

2. The node N to-be-removed has only one child. Here simply removing the node would disconnect the tree, but we can instead bypass it by connecting N's parent to its children (independently of it being in the left or right subtree of N). This won't violate any invariant for the *BST*: for instance, in the case shown in figure 9.15B, N is a left child of its parent P, so it's smaller or equal than P, but likewise, all elements in the subtree rooted at N will be smaller or equal than P, including N's child L.

   If the node N that we need to remove has both children, we can't just replace it with one of its children (for instance, if we were to replace the root in figure 9.15C with its right child R, then we would have to merge R's left child with N's, and that would require worst-case linear time (and also be a waste).

   Instead, we can find the successor[8] of N: by construction, it will be the minimum node in its left subtree, let's call it M, which in turn means the leftmost node of its right subtree. Once we have found it, we can delete M and replace N's value with M's value. No invariant will be violated, because by definition M will be no smaller than any node in N's left branch (because it was no smaller than N, that was no smaller than any node in its left subtree), and no bigger than any node in N's right branch (because it was its minimum).

   Moreover, M will certainly fall in either case (A) or case (B), because being the left-most node it won't have a